

Online Resource 1

Supplement to “Identifiability and Feasibility Budgets in Pre-Export Row Scaling for Mixed-Integer Programs”

Mathias Rodríguez Castro

Facultad de Ingeniería, Universidad de la República, Montevideo, Uruguay

mathiasr@fing.edu.uy

Prepared for Optimization Letters

This supplement specifies the computational evidence and its limitations. Paths are relative to the accompanying repository. Frozen results can be checked without an optimizer license. The earlier draft and intermediate runs are retained to make corrections traceable; they are not interchangeable with the final datasets.

S1 Proof for the triangular model

For $R \in \mathbb{R}^{m \times n}$, $m, n \geq 1$, set $\rho = \|R\|_2$ and

$$C_\gamma = \begin{bmatrix} I_n & 0 \\ \gamma R & \gamma I_m \end{bmatrix}, \quad \gamma > 0.$$

The main-text proposition states that the unique optimal common coupling factor is $(1 + \rho^2)^{-1/2}$ and the minimum condition number is $\sqrt{1 + \rho^2} + \rho$. The interval-constrained optimum clips this factor to the interval.

Proof. An SVD of R and orthogonal transformations reduce the nontrivial spectrum to blocks $\begin{bmatrix} 1 & 0 \\ \gamma\sigma & \gamma \end{bmatrix}$, one for each singular value σ of R , with possible additional singular values 1 or γ . Each block’s squared singular values have product γ^2 and sum $1 + \gamma^2(1 + \sigma^2)$. Their largest value increases and smallest value decreases with σ ; the block at $\sigma = \rho$ therefore determines both extrema, including any additional values. If k is their singular-value ratio, then

$$k + k^{-1} = \frac{1 + \gamma^2(1 + \rho^2)}{\gamma} = \gamma^{-1} + \gamma(1 + \rho^2).$$

The left-hand side is increasing for $k \geq 1$. The right-hand side has a unique minimum at $\gamma_* = (1 + \rho^2)^{-1/2}$, decreases before that point, and increases after it. At the minimum, $k + k^{-1} = 2\sqrt{1 + \rho^2}$, whose root $k \geq 1$ is $\sqrt{1 + \rho^2} + \rho$. The interval result follows from the same monotonicity. For $\rho = 0$, the argument reduces to balancing the diagonal entries at one. $\square$

This scalar result assumes already whitened local blocks and one common coupling factor. It does not solve general diagonal preconditioning and says nothing directly about simplex bases or mixed-integer search.

Table S1: Reconstructed operational campaign. Arithmetic mean PAR10 in seconds on the 15 assigned instances per cell; a timeout contributes 18,000 seconds. Parentheses give completed runs. All rows concern Gurobi.

Class	Gap	Base	Flat-GM	Flat-L2	Role-Hybrid
Simple	1%	11.90 (15)	2.12 (15)	20.44 (15)	2.11 (15)
Simple	0.1%	52.00 (15)	72.98 (15)	33.17 (15)	72.63 (15)
SG-Ter-Mer	1%	116.15 (15)	1216.51 (14)	1217.60 (14)	1217.34 (14)
SG-Ter-Mer	0.1%	134.08 (15)	1216.24 (14)	1216.05 (14)	1216.91 (14)

S2 Operational evidence and reconstruction

The final historical campaign assigned 15 instances in each of two dispatch families, four policies, and two relative MIP gaps. Thus there are 240 assigned Gurobi 13.0.1 runs (Debian GNU/Linux 12). All use seed 42, 16 threads and a 1,800-second limit. The four policies are Base, Flat-GM, Flat-L2 and Role-Hybrid. The analysis retains the assigned subsets and includes every non-completion in the primary runtime endpoint.

For assigned instance i in a cell, let t_i be the recorded runtime and define $T_i = t_i$ for completion and $T_i = 18,000$ otherwise. The primary summary is $15^{-1} \sum_i T_i$. This PAR10 endpoint measures operational completion under the specified stopping criteria. It is not an exact primal or optimality certificate. In particular, a solver-completed point with a concerning original residual is still counted as completed, with feasibility diagnostics reported separately.

Work and uncertainty. The old runner’s scaled-policy Work markers inherited a fixed, zero-decimal stream format. Native Gurobi summaries instead retain two decimals. The reconstruction uses those native summaries for every policy, at their common 0.01-unit resolution. It never combines a Work value with a timeout penalty measured in seconds. Byte equality of rounded Work or node counts cannot prove equality of exported models.

Secondary Work ratios use only the instances completed by every policy in a given family/gap cell. Ratios are paired by instance; their geometric mean is $\exp(n^{-1} \sum_i \log(W_{i,p}/W_{i,\text{Base}}))$. Percentile intervals use 10,000 paired bootstrap samples, seed 20260908. These are exploratory intervals, without multiplicity adjustment, and are conditional on the common completion pool. A separate sensitivity calculation uses the intervals of width 0.01 implied by rounding each native Work value. Neither calculation accounts for the excluded timeout’s unobserved completion effort.

Flat-GM/Base on the 14 common completed SG-Ter-Mer instances has ratios 0.698 with interval $[0.486, 0.944]$ at 1% gap, and 0.682 with $[0.470, 0.925]$ at 0.1%. However, instance077 completes under Base and times out under all three scaled policies at both gaps. For Simple at 1%, Flat-L2/Base has Work ratio 1.792 with interval $[1.033, 3.542]$. These conditional observations do not establish a uniform solver advantage. Table S1 gives the complete PAR10 summary.

Original residual normalization. For an equality, the absolute residual is $|a_r^T x - b_r|$. The RHS-normalized residual divides by $1 + |b_r|$, whereas the activity-normalized residual divides by $\max\{1, |b_r|, \sum_j |a_{rj} x_j|\}$. For inequalities the positive violation replaces the absolute equality residual. Neither is a supplied engineering acceptance budget. For caso46e, Flat-L2 at 1% produces an absolute equality residual about 4.97 on a row with zero RHS. Its RHS-normalized value is still 4.97, while the run’s maximum activity-normalized residual is 1.15×10^{-7} . Calling the former a 497% physical error would be unjustified.

The directory `results-revision/research-audit/operational/` contains native cluster logs, the reconstructed run table, cell summaries, paired contrasts, original input hashes and analysis settings. The script `scripts/audit_operational_evidence.py` generates these summaries and the main-text table. A separate two-run smoke check under `precision-smoke/` verifies that the corrected C++ Work marker preserves the solver API’s precision; it does not replace any of the 240 historical observations.

S3 Fresh matrix audit

Ten seeds, 20260909–20260918, are crossed with exponent ranges $[-R, R]$ for $R \in \{1, 2, 3\}$, two disturbance severities (3 and 6), and three coupling patterns: `coupling`, `coupling_uniform`, and `mixed`. There are 180 instances and six exports per instance: Base, SA-Mat, Flat-GM, Flat-L2, Role-Hybrid and SA-Mat-permB. A solver-free DummyLp backend writes the 1,080 LP files.

The parser compares named rows and columns, coefficient supports, row senses, right-hand sides, and the remaining objective/domain sections. It infers each row’s positive factor from a nonzero pivot coefficient and checks the entire row and RHS at relative tolerance 10^{-13} . All final exports pass. This remains a floating-point proportionality check; it is not a rational proof of exact scaling. Byte hashes supply the stronger literal-identity claims where they hold.

The numerical reference rank is determined from row-L2-normalized Base using a relative threshold 10^{-12} . Every variant then uses that same rank for the spectral diagnostic. Independent rank thresholds on each export disagree on 25 of 1,080 cases. Very large unscaled condition estimates near binary64 resolution are therefore not interpreted quantitatively. The paired scaled diagnostics are below 6.5×10^6 .

The authoritative directory is `results-revision/research-audit/identifiability-final/`. Its manifest records the generator binary and script hashes, settings, named comparisons and all exports. The retained `identifiability-confirmatory/` directory used the former fixed precision serializer and is superseded for current matrix claims.

S4 Frozen binary stress protocol

The experiment crosses three binary families, eight seeds (100–107), and four coefficient exponents (0, 3, 6, 9): 96 models with 12 binary variables each. Each model is run under two presolve settings, two solvers, and nine policies, giving 3,456 planned configurations. Budget-round abstains on 288 configurations before calling an optimizer; hence there are 3,168 actual calls. These configurations are not statistically independent observations.

For seed s , NumPy’s default generator is seeded by $20260908 + s$. The cardinality family has one all-ones row. Packing uses three rows of uniformly sampled integers 1 through 9. Signed rows use integers from -9 through 9 , replacing zeros by one. For row r , $k_r = \max\{2, \lfloor 0.55 \sum_j \max(W_{rj}, 0) \rfloor\}$; profits are uniform integers 1 through 30. The represented constraints are $10^e Wx \leq 10^e k - 0.11$. Integer enumeration proves the binary optimum for each stored model. Changing e also changes the relaxation; it is not pure row re-emission. The budget is 0.01 in the declared canonical stress-test units.

Both solvers use one thread, the instance seed, a ten-second limit and zero target absolute and relative MIP gaps. Primal and integrality tolerances are 10^{-6} , except GM-tight, which uses 10^{-9} . Gurobi’s enabled-presolve arm uses its automatic presolve setting; HiGHS uses presolve on. Internal solver scaling remains at its default. Recorded versions are Python 3.10.12, NumPy 2.2.6, HiGHS 1.15.1 and Gurobi 13.0.2. Solving uses the Python APIs rather than text-file import.

Table S2: Enumerated stress experiment, 192 planned configurations per policy and solver. “Verified” means that the returned point passes the declared acceptance rule and its rounded assignment attains the independently enumerated optimum. “Other” includes unverified outputs and solver errors; “Abstain” means no optimizer call was attempted. Abstention is not a performance success.

Method	HiGHS 1.15.1			Gurobi 13.0.2		
	Verified	Other	Abstain	Verified	Other	Abstain
Base	167	25	0	156	36	0
GM	141	51	0	140	52	0
L2	141	51	0	139	53	0
GM-tight	166	26	0	166	26	0
Dyadic-GM	142	50	0	140	52	0
Budget-only	191	1	0	151	41	0
Budget-GM	182	10	0	152	40	0
Budget-dyadic	161	31	0	153	39	0
Budget-round	48	0	144	48	0	144

Base, GM and L2 scale all rows according to their stated kernels, without a skip band. Budget-only uses ϵ/δ_r . Budget-GM projects GM onto the joint budget/coefficient interval with window $[10^{-9}, 10^9]$. Budget-round additionally reserves $10^{-6} \sum_j |a_{rj}|$ for rounding and rejects the whole model if any row’s contract is incompatible. Dyadic-GM imposes the coefficient window and exact binary64 multiplication, including the RHS; Budget-dyadic also imposes the residual budget. The latter two policies were added after the initial floating-endpoint audit and are exploratory, not a new holdout experiment.

The sequence of preserved datasets under `results-revision/research-audit/` is:

Directory	Interpretation
<code>budget-pilot</code>	Single exploratory seed 0; six initial policies.
<code>budget-confirmatory</code>	Seeds 100–107; seven policies; original nearest-rounded endpoint.
<code>budget-confirmatory-v2</code>	Same seven policies and seeds; exact endpoints rounded inward.
<code>budget-final</code>	Authoritative nine-policy run; inward endpoints and two exploratory dyadic controls.

An attractive result from an intermediate implementation is not substituted for the corrected final result. In particular, the earlier HiGHS Budget-GM count of 191 verified configurations becomes 182 with correctly inward-rounded endpoints.

S5 Independent verification and arithmetic checks

The independent script `scripts/research/verify_saved_experiment.py` imports neither the experiment runner nor an optimizer. For each of the 96 stored models it enumerates all 4,096 binary assignments using integer activities, checks the reported optimum, and then recomputes the acceptance decision from the saved point. It checks all 3,456 distinct Cartesian-product configuration keys, all 288 abstentions, factor admissibility, and the 768 dyadic exports with exact rational multiplication.

A verified optimum requires an exactly feasible rounded binary point attaining the enumerated objective, a returned point within 10^{-9} of that rounded point, and bounds respected within 10^{-9} . Raw-point residual budgets and integrality errors are separate diagnostics. In particular, the

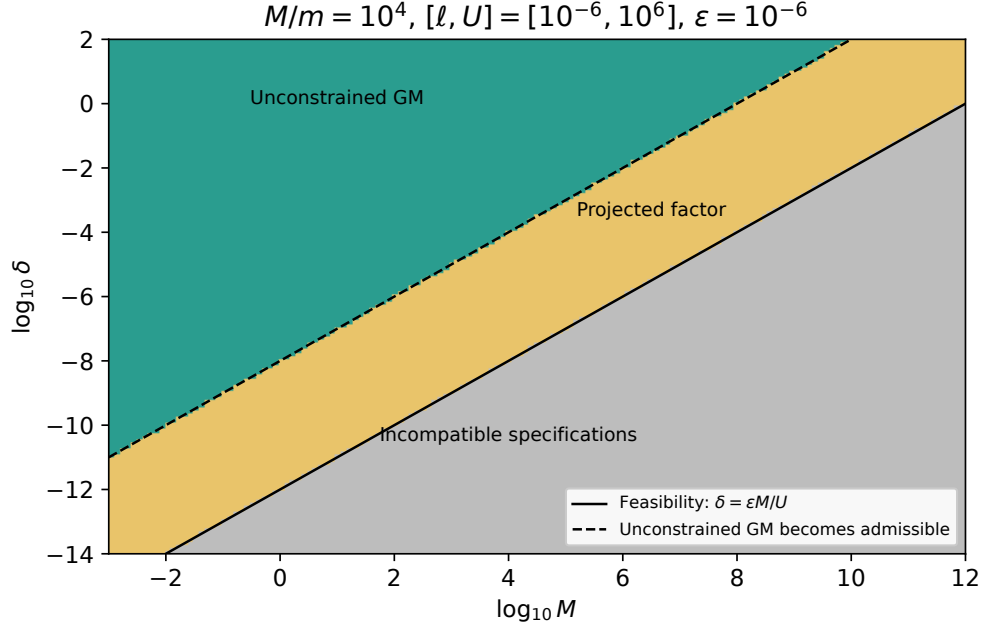


Figure S1: Exact compatibility regions for a row with fixed within-row spread. Below the solid line, the residual budget conflicts with the coefficient ceiling. In the middle region the constrained minimax factor is projected onto the admissible interval; above the dashed line ordinary GM already satisfies it. This diagram concerns the conditional row contract, before reserving any integer-rounding allowance.

verified-optimum label concerns the rounded assignment; it does not claim exact feasibility of the raw floating-point point. A failure of this conservative check does not by itself identify a solver bug.

Rational checks interpret each stored binary64 datum exactly. They do not recover unrecorded real-valued physical data. Arbitrary-factor multiplication may change the represented row by rounding even when its factor lies in the exact admissible interval. The dyadic routine avoids this by intersecting the coefficient/budget interval with exact exponent restrictions on every coefficient and the RHS. It compares candidate centering objectives as rational numbers, because floating logarithms can misclassify a near-midpoint as a tie.

Tests compare continuous interval projection with an independent linear program in log coordinates, check positive re-emission, incompatible specifications, subnormal and overflow boundaries, and a cancellation example where a floating-point residual misses an exact unit violation. A native C++ integration test exercises the actual LP writer with very small coefficients, RHS, bounds and objective values. The serializer uses `max_digits10` and general decimal notation so binary64 values survive decimal round-trip parsing. Serialization correctness and exact multiplication are separate conditions; solver-internal transformations remain outside both guarantees.

S6 Reproduction and artifact layout

The root README and `environment/research-requirements.txt` specify the recorded analysis environment. The following commands need no optimizer license:

```
make research-check-python # tests, saved binary audit, matrix hashes
make research-check # also builds/tests the native C++ exporter
```

```
make research-analysis # current tables and vector figures
make paper # article and this supplement
```

The full export check needs GNU Make and a C++17 compiler. PDF compilation uses pdfLaTeX, BibTeX and standard TeX packages, without the old proprietary-font requirements. Figures are generated with matplotlib from stored data or explicit mathematical expressions.

Fresh experiments write to separate directories and are not required for verification:

```
make research-grid GRID_OUT=results-revision/research-audit/reruns/grid
make research-budget BUDGET_SOLVERS=highs \
  BUDGET_OUT=results-revision/research-audit/reruns/highs
```

The default two-solver budget rerun also requires Gurobi and a suitable license. Runtime measurements need not reproduce bit for bit across hardware or solver versions. Source snapshots and hashes in the final budget directory identify the implementation that generated the reported results; later arithmetic edge-case repairs are recorded separately, with a comparison against all saved row factors. The accompanying source/data archive (Online Resource 2) preserves the inputs needed to regenerate the current main tables and figures.